

C++ PROJECT BUILDER ROADMAP

A practical 6-month system to build projects, develop problem-solving thinking, and become confident at turning ideas into working software.

Core principle: Don't chase big projects. Build small systems, learn reusable ideas, then combine them.

Use this as a roadmap, not a syllabus. The goal is independence: taking an unclear problem, breaking it down, implementing it, debugging it, testing it, and shipping it.

1. THE END GOAL

After six months, you should be able to receive a problem such as “build a local CLI music manager” and independently turn it into requirements, modules, data structures, algorithms, file handling, system interaction, testing, and a usable final program.

Skill stack: C++ fundamentals → STL → OOP → files/filesystem → data structures → algorithms → debugging → OS/process concepts → external libraries → architecture → documentation → iteration.

Your progress is not the number of projects. It is how independently you can solve the next project.

2. SIX-MONTH ROADMAP

Month 1 — Build the basics

- Projects: Number Guessing Game, Calculator, Quiz CLI, Contact Book, Expense Tracker.
- Focus: functions, loops, strings, vectors, maps, structs/classes, input handling.
- Each project: roughly 1–4 days. Do not polish endlessly.
- Outcome: turn a simple requirement into a working CLI program.

Month 2 — OOP + persistence

- Projects: Student Record Manager, Library Manager, Bank Account Simulator, Inventory Manager.
- Focus: classes, constructors, encapsulation, composition, STL containers, file I/O.
- Add save/load to at least two projects.
- Outcome: model real-world systems and persist data.

Month 3 — Filesystem + data

- Projects: File Explorer CLI, MP3 Finder, Duplicate File Detector, Log Analyzer, CSV Analyzer.
- Focus: `std::filesystem`, paths, maps, sets, sorting, searching, frequency counting.
- Start an edge-case list for every project.
- Outcome: work with real files and datasets.

Month 4 — Algorithms + problem solving

- Projects: Text Search Engine, Mini Autocomplete, Route/Graph Explorer, Task Scheduler Simulator, Cache Simulator.
- Focus: complexity, searching, hashing, stacks, queues, linked lists, trees/graphs where appropriate, BFS/DFS.
- Daily DSA: one problem on most study days. Explain the approach in your own words.

- Outcome: recognize patterns instead of treating every problem as new.

Month 5 — System-oriented C++

- Projects: Command Executor, Process Launcher, Mini Shell, Local Notes CLI with indexing, Audio Player Wrapper.
- Focus: OS concepts, processes, commands, error handling, platform differences, external libraries.
- Build each subsystem separately before combining it.
- Outcome: understand how C++ interacts with the OS and external components.

Month 6 — Capstone: CLI Music Manager

- v0.1 scan/list → v0.2 search/library → v0.3 playlists → v0.4 queue/next/previous → v0.5 playback integration → v0.6 persistence/settings → v1.0 testing/documentation.
- Focus: architecture, interfaces, debugging, UX, Git history, README and demo.
- Outcome: one serious portfolio project built from smaller components.

3. PROJECT LADDER: FROM SIMPLE TO MUSIC PLAYER

#	Mini project	Main skill	Purpose
1	Folder/File Lister	filesystem + paths	Feeds the later MP3 scanner.
2	MP3 Finder	extension filtering + filesystem	Scans only .mp3 files.
3	Song Catalog	vector + struct/class	Stores song paths/metadata.
4	Song Search	strings + algorithms	Finds songs by name.
5	Song Class	OOP	Makes a song a reusable object.
6	Music Library Class	OOP + STL	Centralizes add/remove/search/list.
7	Playlist Manager	vector + OOP	Creates and edits playlists.
8	Playlist Save/Load	file I/O	Makes data survive restarts.
9	Queue Manager	queue/deque + state	Models "up next".
10	Shuffle/Repeat Engine	randomization + state	Adds playback behavior.
11	Command Executor	OS interaction	Runs an external command/program.
12	Process Launcher	process concepts	Learns process launching.
13	Audio File Opener	OS + paths	Opens a local audio file.
14	Audio Playback Wrapper	external audio library	Creates play/stop/pause-style functions.
15	Integrated CLI Music Manager	architecture + all above	Combines the pieces.

4. THE PROJECT-BUILDING LOOP

Use the same loop every time. It prevents the "I don't know where to start" freeze.

1. Define: "I want to help __ do __."

2. MVP: choose the smallest useful version. Music Player v0.1 = scan folder + list MP3 files. Nothing else.

3. Inputs/outputs: what enters, what comes out, what can fail?

4. Modules: split the system into small responsibilities.

5. Data structures: decide whether each problem needs vector, map, set, queue, stack, graph, or a custom class.

6. Prototype: prove the riskiest part first.

7. Implement: one feature at a time, compile/run often.

8. Test: normal case + empty input + invalid input + boundary + missing data + duplicates.

9. Refactor: improve names, responsibilities, duplication and class boundaries.

10. Ship: README, build/run steps, demo, limitations and future work.

5. THINKING DEVELOPMENT

When you see a problem, don't immediately code. First create a mental map.

7-question method:

- What exactly is the problem?
- Who is the user?
- What is the smallest useful solution?
- What data must exist?
- What operations must happen to that data?
- What can fail?
- Which part is hardest or most uncertain?

Draw: Input → Processing → Data → Output. If you cannot draw this, the problem is probably still too vague.

Example: Folder path → filesystem traversal → filter extension == .mp3 → store paths in vector → print indexed list.

6. HOW TO STOP GETTING STUCK

Getting stuck is normal. Staying stuck without changing your approach is the problem.

30-minute rule: try yourself first, then search intelligently.

Debug ladder:

- Reproduce the bug.
- Read the exact compiler/runtime error.
- Inspect the relevant state.
- Reduce to the smallest failing example.
- Test one hypothesis at a time.
- Search the exact error/behavior.
- Read documentation/examples.
- Implement the fix yourself.
- Retest the original case plus an edge case.
- Write why the bug happened.

Golden rule: Never copy a fix you cannot explain. If you used a solution, close it and rewrite the idea yourself.

7. PROBLEM-SOLVING TRAINING

- 4-5 days/week: one DSA/problem-solving question.
- 2-3 sessions/week: 60-90 minutes of project building.
- 1 session/week: improve an old project.
- 1 session/week: explain a technical concept aloud or record a short explanation.
- Every 2 weeks: finish and ship something small.

For every DSA problem record: pattern → brute force → optimized idea → complexity → mistake → final insight.

Do not chase counts blindly. Deep understanding beats copied solutions.

8. PROJECT DIFFICULTY RULE

Use a 70/20/10 learning ratio:

- 70% familiar: you understand most tools.
- 20% new: one or two concepts stretch you.
- 10% uncertain: one genuinely difficult component.

If a project is mostly unknown, shrink it. If everything is familiar, add one meaningful challenge.

9. CODE QUALITY CHECKLIST

- Functions have one clear responsibility.
- Names explain intent.
- Classes represent meaningful entities.
- Input validation exists.
- Errors are handled.
- No giant main().
- Avoid unnecessary globals.
- Extract repeated logic.
- Handle paths/resources carefully.
- Comments explain why, not obvious syntax.
- Write build instructions.
- Use meaningful Git commits.

10. GIT + PORTFOLIO

- Create a repository from the beginning.
- Commit after meaningful milestones.
- README: Problem → Features → Concepts → Build/Run → Usage → Demo → Limitations → Future work.
- Label versions clearly.
- Write a short postmortem: what failed, what you learned, what you would redesign.

11. WEEKLY TEMPLATE

Day	Focus	Time
Mon	DSA + project feature	60-90 + 60 min
Tue	Project + debugging	90-120 min
Wed	DSA + explain what you learned	60 + 20 min
Thu	Project building	90-120 min
Fri	DSA + refactor/test	60 + 45 min
Sat	Long build + README/Git	2-3 h
Sun	Review + next-week plan	30-45 min

12. MOVE-ON CRITERIA

- You can explain the project without opening the code.
- You can rebuild its core logic.
- You can identify 3+ edge cases.

- You can explain your data-structure choices.
- README is complete.
- Git repository is updated.
- You can name one redesign/improvement.

If these are true, move on. Don't spend weeks polishing a tiny CLI project.

13. CAPSTONE: CLI MUSIC MANAGER

Suggested modules:

- CLI: menu, commands, validation.
- Scanner: discover local audio files.
- Song model: path/title/optional metadata.
- Library: add/remove/search/list.
- Playlist: create/edit/save/load.
- Queue: next/previous/up-next.
- Playback adapter: hides the chosen audio implementation.
- Persistence: settings and saved playlists.
- Utilities: errors, logging, path helpers.

Architecture idea: the CLI should not know how audio is played. It should call a Player module/interface. This separation lets you change the playback implementation without rewriting the whole application.

Standard C++ does not itself provide MP3 decoding/playback. Use an appropriate multimedia/audio library or system-player integration and check its current official documentation and licensing.

14. MASTER CHECKLIST

- Finish 5 Month-1 mini projects.
- Finish 4 Month-2 OOP projects.
- Finish 5 Month-3 filesystem/data projects.
- Maintain DSA practice.
- Build 3 algorithmic mini-projects.
- Build one OS/process mini-project.
- Prototype single-file audio playback.
- Build Music Manager v0.1.
- Build v0.2–v0.6.
- Write README + architecture notes.
- Create a demo/video.
- Write the final postmortem.

15. RULES TO REMEMBER

- Don't wait until you “know enough”.
- Start with the smallest version.
- Break every big feature into testable problems.

- Use documentation as a tool.
- Debug systematically.
- Finish more often than you restart.
- Build fewer projects deeply.
- Every project must teach a reusable idea.
- Aim for independence, not memorization.
- Keep a problem log. Your mistakes become future knowledge.

FINAL NOTE

You do not become pro by suddenly learning everything. You become strong at problem solving by repeatedly taking something unclear, making it concrete, breaking it into pieces, building the smallest version, debugging it, testing it, and finishing it.

Start with Project #1. Do not jump straight to the Music Player until the earlier projects have taught you why its architecture exists.

Reference: C++ language/library reference material: cppreference.com. For external libraries, consult their current official documentation before implementation.